BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

Veritabanı (Database)

EBRU AYDOĞAN

Veritabanı (Database) Nedir?

- Veritabanı, bilgilerin sistematik bir şekilde depolandığı, yönetildiği ve güncellendiği dijital bir depo alanıdır.

BTK AKADEMİ

Veritabanı (Database) Nedir?

- Şu ana kadar kullandığımız JSON veya CSV dosyaları küçük işler için harikadır; ancak veri büyüdüğünde (binlerce satır olduğunda) arama yapmak ve veriyi güvenle saklamak imkansız hale gelir.
- Neden Kullanılır?
  - **Hız:** Milyonlarca kayıt içinden saniyeler içinde arama yapabilir.
  - **Güvenlik:** Veriye kimin erişebileceğini kontrol eder.
  - **Bütünlük:** Hatalı veri girişini (örneğin fiyat kısmına harf yazılmasını) engelleyebilir.

Veritabanı (Database) Nedir?

- Piyasada farklı ihtiyaçlara göre özelleşmiş veritabanları vardır:
  - **Dev Sistemler:** MySQL, PostgreSQL, Oracle, Microsoft SQL Server (Büyük şirketler, bankalar kullanır).
  - **Gömülü Sistemler: SQLite** (Mobil uygulamalar, tarayıcılar ve bizim gibi eğitim projeleri kullanır).
- **Neden SQLite Seçtik?**
  - Kurulum gerektirmez (Python ile hazır gelir).
  - Şifre, kullanıcı adı veya sunucu ayarı istemez.
  - Veritabanınız sadece bilgisayarınızdaki tek bir .db dosyasıdır. İstediğiniz yere kopyalayıp götürebilirsiniz.

Veritabanı Kavramları

- Veritabanı aslında bir Excel dosyasına benzer ama çok daha akıllıdır:
  - **Tablo (Table):** Verilerin kategorize edildiği ana grup (Örn: Ogrenciler tablosu).
  - **Sütun (Column / Field):** Saklanacak verinin türü (Örn: Ad, Soyad, Numara).
  - **Satır (Row / Record):** Her bir kişiye veya işleme ait tek bir veri seti (Örn: 1 numaralı satırda "Ali Yılmaz" bilgisi).
  - **Primary Key (Birincil Anahtar):** Her satırı birbirinden ayıran benzersiz kimlik numarası (TC No veya Öğrenci No gibi).

Python ile SQLite Bağlantısı

- Python'da veritabanı işlemleri için sqlite3 modülünü kullanırız. İşlem sırası.

```python
import sqlite3

# 1. Veritabanına bağlanıyoruz (Dosya yoksa o an oluşturulur)
connection = sqlite3.connect("btk_akademi.db")

# 2. Cursor (İmleç) oluşturuyoruz.
# Bu imleç, SQL komutlarımızı veritabanına taşıyan "elçimiz"dir.
cursor = connection.cursor()

print("Veritabanı başarıyla oluşturuldu ve bağlantı kuruldu!")

# 3. İşimiz bitince kapıyı kapatıyoruz
connection.close()
```

BTK AKADEMİ

SQL Dilinde Tablo Oluşturma (CREATE TABLE)

- Veritabanına "Hangi bilgileri saklayacağız?" bilgisini vermemiz gerekir. Bunun için SQL (Structured Query Language) dilini kullanırız.
- **Veri Tipleri:** TEXT (Metin), INTEGER (Tam sayı), REAL (Ondalıklı sayı).

BTK AKADEMİ

```python
import sqlite3

db = sqlite3.connect("btk_akademi.db")
cursor = db.cursor()

# Tablo oluşturma komutu
# IF NOT EXISTS: "Eğer bu tablo zaten varsa hata verme" demektir.
cursor.execute("""
    CREATE TABLE IF NOT EXISTS urunler (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        urun_adi TEXT NOT NULL,
        fiyat REAL,
        stok_adedi INTEGER
    )
""")

# commit(): Yapılan bu değişikliği veritabanı dosyasına kalıcı olarak işle.
db.commit()

print("Ürünler tablosu hazır!")
db.close()
```

BTK AKADEMİ

SQL Dilinde Tablo Oluşturma (CREATE TABLE)

- Neden commit() Yapıyoruz?
  - Python ile bir tablo oluşturduğunuzda veya veri eklediğinizde bu işlem önce "geçici hafızaya" alınır.
  - Eğer db.commit() demezseniz, program kapandığında yaptığınız değişiklikler veritabanı dosyasına yazılmaz.
  - Bu, "Dosyayı Kaydet" butonuna basmak gibidir. SELECT (Veri çekme) işlemlerinde commit gerekmez, ama veriyi değiştiren (CREATE, INSERT, UPDATE) her işlemde **şarttır**.

Veri Ekleme (INSERT INTO)

- Oluşturduğumuz tabloya yeni bir satır (kayıt) eklemek için INSERT INTO komutunu kullanıyoruz.
- **Kritik Kural:** Eğer tablomuzda id sütunu AUTOINCREMENT (otomatik artan) ise, veri eklerken id belirtmemize gerek yoktur; veritabanı onu kendisi atar.
- Değerleri gönderirken Python'daki f-string yapısı yerine ? (placeholder) kullanmak, **SQL Injection** denilen siber saldırıları önlemek için bir dünya standardıdır.

Veri Ekleme (INSERT INTO)

```python
import sqlite3

db = sqlite3.connect("btk_akademi.db")
cursor = db.cursor()

# Tek bir kayıt ekleme
sql = "INSERT INTO urunler (urun_adi, fiyat, stok_adedi) VALUES (?, ?, ?)"
values = ("Kablosuz Mouse", 450.50, 100)

cursor.execute(sql, values)

db.commit() # Değişikliği kaydetmeyi unutmuyoruz!
print(f"{cursor.rowcount} adet kayıt eklendi. Son eklenen ID: {cursor.lastrowid}")

db.close()
```

BTK AKADEMİ

Veri Ekleme (INSERT INTO)

- Çoklu Veri Ekleme (executemany)
  - Elimizde bir liste dolusu veri varsa (örneğin sabahki API'den gelen 50 tane kur bilgisi), bunları tek tek değil, tek bir hamlede eklemek çok daha hızlıdır.
  - Bunun için execute() yerine executemany() metodunu kullanırız.

Veri Ekleme (INSERT INTO)

```python
import sqlite3

db = sqlite3.connect("btk_akademi.db")
cursor = db.cursor()

liste = [
    ("Klavye", 750.0, 50),
    ("Monitör", 3500.0, 15),
    ("USB Kablo", 120.0, 200)
]

sql = "INSERT INTO urunler (urun_adi, fiyat, stok_adedi) VALUES (?, ?, ?)"

# Listenin tamamını tek seferde gönderiyoruz
cursor.executemany(sql, liste)

db.commit()
print(f"{len(liste)} kayıt başarıyla sisteme işlendi.")
db.close()
```

BTK AKADEMİ

Veri Okuma ve Listeleme (SELECT)

- Veritabanındaki verileri geri çekmek için SELECT komutu kullanılır.
- *** İşareti:** "Tüm sütunları getir" demektir.
- Verileri Python içinde kullanabilmek için iki temel metodumuz var:
  - **fetchone()**: Sadece en üstteki tek bir kaydı getirir.
  - **fetchall()**: Tüm sonuçları bir liste olarak döndürür.

BTK AKADEMİ

Veri Okuma ve Listeleme (SELECT)

```python
import sqlite3

db = sqlite3.connect("btk_akademi.db")
cursor = db.cursor()

cursor.execute("SELECT * FROM urunler")

# Verileri çekelim
urunler = cursor.fetchall()

# Gelen veri bir listedir, döngüyle ekrana basalım
print("--- STOK LİSTESİ ---")
for urun in urunler:
    # urun[0]=id, urun[1]=ad, urun[2]=fiyat ...
    print(f"ID: {urun[0]} | Ürün: {urun[1]} | Fiyat: {urun[2]} TL")

db.close()
```

BTK AKADEMİ

Veri Güncelleme (UPDATE)

- Veritabanındaki mevcut bir bilgiyi değiştirmek için kullanılır (Örn: Bir ürünün fiyatının değişmesi veya stok miktarının azalması).
- **Hayati Uyarı:** UPDATE komutu kullanırken mutlaka **WHERE** koşulu eklenmelidir. Eğer WHERE kullanmazsanız, tablodaki **tüm satırlar** aynı yeni değerle güncellenir!
- Genellikle güncelleme işlemleri, kaydın benzersiz olan id numarası üzerinden yapılır.

Veri Güncelleme (UPDATE)

```python
import sqlite3

db = sqlite3.connect("btk_akademi.db")
cursor = db.cursor()

# 1 Numaralı ürünün fiyatını 500.00 yapalım
yeni_fiyat = 500.00
urun_id = 1

sql = "UPDATE urunler SET fiyat = ? WHERE id = ?"
cursor.execute(sql, (yeni_fiyat, urun_id))

db.commit()
print(f"{cursor.rowcount} adet kayıt güncellendi.")
db.close()
```

BTK AKADEMİ

Veri Silme (DELETE)

- Tablodaki belirli bir kaydı tamamen kaldırmak için kullanılır.
- **Kritik Uyarı:** Tıpkı güncellemede olduğu gibi, WHERE koşulu hayati önem taşır. DELETE FROM urunler derseniz, tablodaki tüm verileri tek hamlede silersiniz (Geri dönüşü zordur!).
- Silme işleminden sonra id numaraları kaldığı yerden devam eder (Silinen ID tekrar kullanılmaz).

Veri Silme (DELETE)

```python
import sqlite3

db = sqlite3.connect("btk_akademi.db")
cursor = db.cursor()

# ID'si 3 olan ürünü silelim
target_id = 3

cursor.execute("DELETE FROM urunler WHERE id = ?", (target_id,))

db.commit()
print("Kayıt başarıyla silindi.")
db.close()
```

BTK AKADEMİ

Kayıtları Teker Teker veya Toplu Çekme

- Bazen binlerce kayıt arasından sadece ilkini veya belirli bir miktarını almak isteriz:
  - **fetchone()**: Sorgu sonucundaki ilk satırı bir **Tuple** olarak döner.
  - **fetchmany(n)**: Belirlediğiniz n sayısı kadar kaydı liste olarak döner.
  - **fetchall()**: Tüm sonuç kümesini döner.

Kayıtları Teker Teker veya Toplu Çekme

```python
import sqlite3

db = sqlite3.connect("btk_akademi.db")
cursor = db.cursor()

cursor.execute("SELECT * FROM urunler")

# Sadece ilk kaydı alalım
ilk_urun = cursor.fetchone()
print(f"En Üstteki Ürün: {ilk_urun[1]}") # urun_adi

# Kalanlardan sonraki 2 kaydı alalım
sonraki_ikili = cursor.fetchmany(2)
for u in sonraki_ikili:
    print(f"Sıradaki Ürün: {u[1]}")

db.close()
```

BTK AKADEMİ

Veriyi Filtreleme (WHERE Koşulu)

- Binlerce kayıt arasından sadece belirli şartlara uyanları çekmek için kullanılır.
- SQL'de sadece = değil; >, <, >=, <=, != (eşit değil) gibi operatörler de kullanılabilir.
- **Mantıksal Operatörler:**
  - AND: Her iki şart da doğru olmalı.
  - OR: Şartlardan biri doğru olsa yeterli.

BTK AKADEMİ

Veriyi Filtreleme (WHERE Koşulu)

```python
import sqlite3

db = sqlite3.connect("btk_akademi.db")
cursor = db.cursor()

# Fiyatı 1000 TL'den büyük olan ve stokta olan ürünleri getir
sql = "SELECT * FROM urunler WHERE fiyat > ? AND stok_adedi > 0"
cursor.execute(sql, (1000,))

sonuclar = cursor.fetchall()
for urun in sonuclar:
    print(f"Pahalı Ürün: {urun[1]} - Fiyat: {urun[2]}")
```

BTK AKADEMİ

Metin İçinde Arama (LIKE Operatörü)

- Tam eşleşme değil de, metnin içinde geçen bir kelimeyi aramak için kullanılır.
- % İşareti (Wildcard):
  - 'A%': 'A' ile başlayanlar.
  - '%A': 'A' ile bitenler.
  - '%A%': İçinde 'A' geçenler.

BTK AKADEMİ

Metin İçinde Arama (LIKE Operatörü)

```python
# İsmi 'K' ile başlayan ürünleri ara
search = "K%"
cursor.execute("SELECT * FROM urunler WHERE urun_adi LIKE ?", (search,))

for urun in cursor.fetchall():
    print(f"Bulunan Ürün: {urun[1]}")
```

BTK AKADEMİ

Verileri Sıralama (ORDER BY)

- Gelen verilerin karışık değil, belirli bir sütuna göre dizilmesini sağlar.
- **ASC (Ascending):** Küçükten büyüğe / A'dan Z'ye (Varsayılan).
- **DESC (Descending):** Büyükten küçüğe / Z'den A'ya.

BTK AKADEMİ

Verileri Sıralama (ORDER BY)

```
# Ürünleri fiyata göre pahalıdan ucuza sırala
cursor.execute("SELECT * FROM urunler ORDER BY fiyat DESC")

for urun in cursor.fetchall():
    print(f"{urun[1]} - {urun[2]} TL")
```

BTK AKADEMİ

SQL'de Matematik (Aggregate Fonksiyonlar)

- Verileri Python'a çekip döngüyle toplamak yerine, bu hesabı doğrudan veritabanı motoruna yaptırmak çok daha hızlıdır.
- **Temel Fonksiyonlar:**
  - COUNT(): Satır sayısını sayar.
  - SUM(): Sütundaki değerleri toplar.
  - AVG(): Ortalama alır.
  - MAX() / MIN(): En büyük ve en küçük değeri bulur.

SQL'de Matematik (Aggregate Fonksiyonlar)

```
# Toplam stok miktarını ve ortalama fiyatı bulalım
cursor.execute("SELECT SUM(stok_adedi), AVG(fiyat) FROM urunler")
toplam_stok, ort_fiyat = cursor.fetchone()

print(f"Depodaki Toplam Ürün: {toplam_stok}")
print(f"Ortalama Ürün Fiyatı: {ort_fiyat:.2f} TL")
```

BTK AKADEMİ

Verileri Gruplama (GROUP BY)

- Verileri belirli bir kategoriye göre paketleyip, her grup için ayrı hesaplama yapmak için kullanılır (Örn: Her kategoride kaç ürün var?).
- Genellikle Aggregate fonksiyonları ile birlikte kullanılır.
- Örnek:

```
cursor.execute("SELECT kategori, COUNT(*) FROM urunler GROUP BY kategori")
```